AUSPIFY TECHNOLOGIES

# AUSPIFY — FRONTEND DEVELOPMENT INTERNSHIP

Project Completion & Technical Report


**Prepared by**

Bassim Ghaly

**Organization**

Auspify Technologies


**Position**

Frontend Development Intern

**Internship period**

23 October 2026 – 23 November 2026

Submission-ready technical summary

# Contents


1 Introduction ..... 1
1.1 Scope and accuracy ..... 1
2 Internship Overview ..... 1
3 Project Reports ..... 1
3.1 Task 1 — Responsive Landing Page ..... 1
3.1.1 Key features ..... 1
3.1.2 Technologies used ..... 2
3.1.3 Implementation highlights ..... 2
3.1.4 Project structure and documentation ..... 2
3.1.5 Verification ..... 2
3.2 Task 4 — Weather Dashboard ..... 2
3.2.1 Key features ..... 2
3.2.2 Technologies used ..... 2
3.2.3 Implementation highlights ..... 3
3.2.4 Project structure and documentation ..... 3
3.2.5 Verification ..... 3
3.3 Task 5 — E-Commerce Product Interface ..... 3
3.3.1 Key features ..... 3
3.3.2 Technologies used ..... 3
3.3.3 Implementation highlights ..... 3
3.3.4 Project structure and documentation ..... 4
3.3.5 Verification ..... 4
3.4 Task 6 — Modern React Dashboard ..... 4
3.4.1 Key features ..... 4
3.4.2 Technologies used ..... 4
3.4.3 Implementation highlights ..... 4
3.4.4 Project structure and documentation ..... 4
3.4.5 Verification ..... 4
4 Technical Skills Demonstrated ..... 5
5 Repository Organization ..... 5
6 Final Conclusion ..... 5
7 Submission Checklist ..... 6

# 1 Introduction

This report presents the frontend development work submitted as part of the Auspify Technologies internship. It documents four completed projects covering static web development, API-driven interfaces, e-commerce application structure, and analytics dashboard development.

The projects are maintained in separate repository directories. Each project demonstrates a different level of frontend implementation, from accessible responsive layouts built with HTML, CSS, and JavaScript to React applications with routing, state management, data visualization, and TypeScript.

## 1.1 Scope and accuracy

Technical descriptions in this report are based on the project information and repository documentation supplied with the submission. No build, lint, or automated-test result is claimed unless it was explicitly performed and recorded. The project-specific README files remain the authoritative source for detailed setup instructions.

# 2 Internship Overview

| Item | Details |
|---|---|
| Organization | Auspify Technologies |
| Position | Frontend Development Intern |
| Internship period | 23 October 2026 – 23 November 2026 |
| Completed tasks | 4 out of 6 internship tasks |
| Technical scope | Responsive UI, vanilla JavaScript, React, TypeScript, API integration, routing, state management, forms, validation, charts, accessibility, and responsive design |

The completed work spans both framework-free and framework-based frontend development. The static projects focus on semantic structure, responsive CSS, accessible interaction, and asynchronous API handling. The React projects focus on reusable components, application routing, client-side state, external data, form flows, animation, and visual analytics.

# 3 Project Reports

## 3.1 Task 1 — Responsive Landing Page

### Responsive Landing Page

COMPLETED

**Project overview.** A responsive marketing landing page for Nexora, a fictional workspace product. It is implemented as a static single-page website using HTML5, CSS3, SVG assets, and vanilla JavaScript.

### 3.1.1 Key features

- Responsive header with brand link and desktop navigation.
- Collapsible mobile navigation with open and close controls.

- Skip-to-main-content link for keyboard and screen-reader users.
- Hero, product preview, trust indicators, features, and call-to-action sections.
- Accessible mobile navigation using `aria-expanded`, `aria-controls`, focus trapping, and Escape-key support.
- Responsive breakpoints at 640px, 768px, and 1024px.

### 3.1.2 Technologies used

HTML5, CSS3, CSS custom properties, responsive media queries, vanilla JavaScript, and SVG assets. No framework or build step is documented for this project.

### 3.1.3 Implementation highlights

The implementation emphasizes semantic page structure and accessible navigation behavior. The mobile menu is designed to communicate state to assistive technologies and to support keyboard interaction. The layout uses responsive breakpoints rather than depending on a frontend framework.

### 3.1.4 Project structure and documentation

Source code is located in `Task-1-Landing-Page/`, with project documentation in `Task-1-Landing-Page/README.md`. The project can be opened through `Task-1-Landing-Page/index.html` without package installation.

### 3.1.5 Verification

The supplied documentation describes direct browser execution and does not provide an independent build, lint, or automated-test result. This report therefore records the documented run method rather than claiming additional verification.

## 3.2 Task 4 — Weather Dashboard

**Weather Dashboard** COMPLETED

**Project overview.** A responsive dashboard that searches for cities and displays current conditions, highlights, hourly forecasts, and a seven-day forecast using Open-Meteo services.

### 3.2.1 Key features

- Real-time weather data from the Open-Meteo API without an API key.
- City search through the Open-Meteo Geocoding API, including Enter-key support.
- Current temperature, condition, feels-like value, daily high/low, and local time.
- Weather highlights for humidity, wind, precipitation, sunrise, and sunset.
- Hourly forecast with time, icon, temperature, and precipitation probability.
- Seven-day forecast with day name, date, condition, and high/low values.
- Celsius/Fahrenheit switching without additional API requests.
- Loading, invalid-input, no-results, network-error, API-error, and retry states.

### 3.2.2 Technologies used

Semantic HTML5, CSS3, Flexbox, CSS Grid, media queries, vanilla JavaScript, ES6+ modules, `async/await`, Fetch API, and `AbortController`. Weather data is provided by Open-Meteo.

### 3.2.3 Implementation highlights

The project uses asynchronous data fetching and geocoding as separate steps in the city-search flow. Unit conversion is applied to the existing response data, allowing the user to switch between Celsius and Fahrenheit without requesting the same data again. The interface explicitly accounts for loading, invalid input, missing results, and recoverable request failures.

### 3.2.4 Project structure and documentation

Source code is located in `Task-4-Weather-Dashboard/`, with project documentation in `Task-4-Weather-Dashboard/README.md`. Because the application makes cross-origin requests, the documented local run method uses a small HTTP server:

```
cd Task-4-Weather-Dashboard\npython3 -m http.server 8000
```

### 3.2.5 Verification

The supplied documentation confirms the required local serving method and the external API dependency. No independent build, lint, or automated-test result is claimed here.

## 3.3 Task 5 — E-Commerce Product Interface

**E-Commerce Product Interface** COMPLETED

**Project overview.** A React-based e-commerce frontend covering product discovery, filtering, cart flows, account-related routes, wishlist behavior, forms, notifications, and responsive interface patterns.

### 3.3.1 Key features

- Routes for Home, Products, Cart, Checkout, Login, Register, Profile, Wishlist, and Not Found.
- Protected-route handling for authenticated areas.
- Product listing from an external API through Axios.
- Loading skeletons and API error handling.
- Search, category, brand, and price-range filtering with pagination.
- Clear-all filtering action.
- Redux Toolkit state slices for authenticated and guest carts.
- Light, dark, and system theme modes.
- Wishlist state, Formik/Yup forms, Framer Motion animations, React Toastify notifications, and Swiper carousels.

### 3.3.2 Technologies used

React 19, Vite, Tailwind CSS 4, Redux Toolkit, React Router, Formik, Yup, Axios, Framer Motion, React Toastify, and Swiper.

### 3.3.3 Implementation highlights

The project separates navigation, data access, state management, form handling, and presentation concerns across a modern React application structure. Redux Toolkit is used for cart and wishlist state, while React Router organizes the page-level application flow and protected areas. External product data is handled through a dedicated Axios service layer with loading and error states.

### 3.3.4 Project structure and documentation

The project is located in `Task-5-Ecommerce/`. The documented local workflow is:

```
cd Task-5-Ecommerce\nnpm install\nnpm run dev
```

Production commands are `npm run build` and `npm run preview`. Generated `node_modules/` and `dist/` directories are excluded from version control.

### 3.3.5 Verification

The supplied repository documentation lists the development, build, and preview commands. No independent execution result is claimed in this report.

## 3.4 Task 6 — Modern React Dashboard

**Auspy Insights — React Analytics Dashboard** COMPLETED

**Project overview.** A responsive analytics dashboard designed to demonstrate real API integration, reusable UI components, interactive charts, and a polished SaaS-style interface.

### 3.4.1 Key features

- Responsive analytics dashboard layout.
- Metrics derived from actual responses from DummyJSON public APIs.
- Interactive data visualizations using Recharts.
- Reusable UI components and responsive design patterns.
- TypeScript strict mode for stronger type safety.
- Professional SaaS-oriented visual styling.
- Lucide React icons and code-quality tooling with ESLint and Prettier.

### 3.4.2 Technologies used

React 19, Vite 8, TypeScript 5.8, Tailwind CSS 4, Recharts 3, Lucide React, ESLint 10, `typescript-eslint`, and Prettier 3.

### 3.4.3 Implementation highlights

The dashboard combines typed React components with API-backed metrics and chart components. The use of strict TypeScript, reusable UI elements, and responsive layout rules supports maintainability while keeping the interface suitable for multiple screen sizes.

### 3.4.4 Project structure and documentation

The project is located in `Task-6-React-Dashboard/`. The documented local workflow is:

```
cd Task-6-React-Dashboard\nnpm install\nnpm run dev
```

Production commands are `npm run build` and `npm run preview`.

### 3.4.5 Verification

The supplied repository documentation identifies the development, build, and preview commands. No independent build, lint, or automated-test result is claimed in this report.

# 4 Technical Skills Demonstrated

The four projects demonstrate the following practical frontend capabilities:

- Responsive web design using media queries, Flexbox, CSS Grid, and mobile-first interaction patterns.
- Semantic HTML and accessible interaction, including skip links, ARIA state attributes, keyboard support, and focus management.
- CSS architecture using custom properties, responsive breakpoints, utility styling, and Tailwind CSS.
- Vanilla JavaScript development with modules, asynchronous functions, Fetch API, and `AbortController`.
- React component-based development with Vite and reusable interface components.
- TypeScript development in strict mode for the analytics dashboard.
- API integration and asynchronous data handling with Open-Meteo, DummyJSON, and an external product API.
- Client-side state management with Redux Toolkit and React context-based theming.
- Routing and protected application areas with React Router.
- Form handling and validation with Formik and Yup.
- Interactive data visualization with Recharts.
- Loading, empty, invalid-input, network-error, API-error, and retry states.
- UI/UX implementation through responsive layouts, themes, notifications, animations, and carousels.
- Repository organization and project documentation through task-specific directories and README files.

# 5 Repository Organization

The repository organizes each completed task in its own directory. This keeps source code, project-specific assets, and local run instructions close to the implementation they describe.

```
.\n├── Task-1-Landing-Page/\n├── Task-4-Weather-Dashboard/\n├── Task-5-Ecommerce/
\n├── Task-6-React-Dashboard/\n├── Screenshots/\n├── Submission-Report.pdf\n├──
README.md\n└── .gitignore
```

The root README provides the repository overview and links to task-level documentation where available. The project directories contain the implementation source code and their respective setup instructions.

# 6 Final Conclusion

This submission documents four completed frontend projects covering a broad practical range: accessible static interfaces, API-driven data presentation, React e-commerce application structure, and a typed analytics dashboard. Together, the projects demonstrate experience with responsive design, modern JavaScript and React development, external data, state management, forms, visualization, error handling, and technical documentation.

The work provides a structured record of the internship progress and the technical areas practiced during the period from 23 October 2026 to 23 November 2026.

# 7 Submission Checklist

| Status | Submission item | Reference |
|---|---|---|
| √ | Task 1 — Responsive Landing Page | `Task-1-Landing-Page/` |
| √ | Task 4 — Weather Dashboard | `Task-4-Weather-Dashboard/` |
| √ | Task 5 — E-Commerce Product Interface | `Task-5-Ecommerce/` |
| √ | Task 6 — Modern React Dashboard | `Task-6-React-Dashboard/` |
| √ | Source code and relevant project documentation | Task directories and README files |
| √ | Supporting screenshots, if included | `Screenshots/` directory |
| √ | Final internship report | `Submission-Report.pdf` |

Checklist status is based on the repository structure and project documentation supplied for this report.